

Mini-labo 2 : Routeur / Firewall :

Objectif :

- Séparer deux sous-réseaux (LAN1 et LAN2)
- Faire transiter le trafic via une machine virtuelle jouant le rôle de routeur / firewall
- Mettre en place des règles afin d'autoriser, bloquer ou filtrer certains flux

Prérequis :

- Avoir réalisé le projet Mini-labo précédent
- Disposer de trois machines virtuelles sous Linux (Debian ou Ubuntu)
- La troisième machine virtuelle doit posséder deux cartes réseau

Rappel :

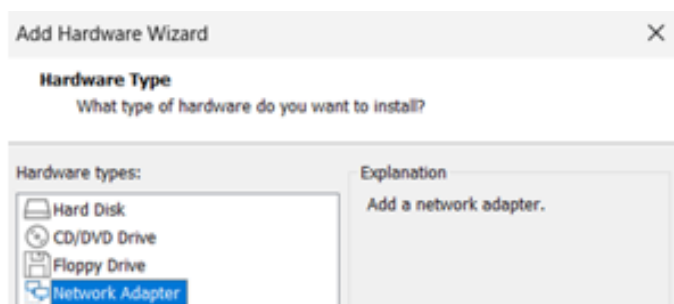
1- Réseau LAN :

Un réseau LAN est un réseau local privé, non connecté directement à Internet.

Nous allons configurer nos deux machines virtuelles sur deux LAN différents afin de simuler deux sous-réseaux distincts.

2- Ajout d'une carte réseau :

La troisième machine virtuelle doit posséder deux cartes réseau afin de pouvoir communiquer avec les deux sous-réseaux.



Chaque carte réseau sera connectée à un LAN différent.

Une fois la deuxième carte réseau installée, il faut connecter :

- Une carte réseau au LAN de la VM1
- Une carte réseau au LAN de la VM2

3- Droits sudo :

Nous attribuons les droits sudo à l'utilisateur afin de pouvoir effectuer les configurations nécessaires.

Etapes :

1- Activation du routage IP :

Le paramètre : `net.ipv4.ip_forward=1` permet d'activer le routage IP sur une machine Linux.

Lorsque cette valeur est définie à 1, le système est autorisé à faire transiter les paquets entre différents réseaux.

```
user@Mini-labo2-firewall:~$ sudo sysctl -w net.ipv4.ip_forward=1
[sudo] Mot de passe de user :
net.ipv4.ip_forward = 1
```

Il est également possible de rendre cette configuration permanente.

```
user@Mini-labo2-firewall:~$ su -
Mot de passe :
root@Mini-labo2-firewall:~# nano /etc/sysctl.conf
root@Mini-labo2-firewall:~#
```

```
# Uncomment the next line to enable packet forwarding for IPv4
net.ipv4.ip_forward=1

# Uncomment the next line to enable packet forwarding for IPv6
# Enabling this option disables Stateless Address Autoconfiguration
# based on Router Advertisements for this host
#net.ipv6.conf.all.forwarding=1

#####
# Additional settings - these settings can improve the network
# security of the host and prevent against some network attacks
# including spoofing attacks and man in the middle attacks through
# redirection. Some network environments, however, require that these
# settings are disabled so review and enable them as needed.
#
# Do not accept ICMP redirects (prevent MITM attacks)
#net.ipv4.conf.all.accept_redirects = 0
#net.ipv6.conf.all.accept_redirects = 0
#_or_
# Accept ICMP redirects only for gateways listed in our default
# gateway list (enabled by default)
#net.ipv4.conf.all.secure_redirects = 1
```

2- Configuration du firewall :

Pour configurer le firewall, nous utilisons **nftables**.

Nous commençons par vérifier que le service est bien installé, puis nous l'activons et le démarrons.

```
user@Mini-labo2-firewall:~$ sudo systemctl status nftables
○ nftables.service - nftables
   Loaded: loaded (/lib/systemd/system/nftables.service; disabled; preset: enabled)
   Active: inactive (dead)
     Docs: man:nft(8)
           http://wiki.nftables.org
```

```
user@Mini-labo2-firewall:~$ sudo systemctl enable nftables
Created symlink /etc/systemd/system/sysinit.target.wants/nftables.service → /lib/systemd/system/nftables.service.
```

```
user@Mini-labo2-firewall:~$ sudo systemctl start nftables
user@Mini-labo2-firewall:~$ sudo systemctl status nftables
● nftables.service - nftables
   Loaded: loaded (/lib/systemd/system/nftables.service; enabled; preset: enabled)
   Active: active (exited) since Thu 2025-10-02 14:35:32 CEST; 9s ago
     Docs: man:nft(8)
           http://wiki.nftables.org
   Process: 2537 ExecStart=/usr/sbin/nft -f /etc/nftables.conf (code=exited, status=0/SUCCESS)
   Main PID: 2537 (code=exited, status=0/SUCCESS)
      CPU: 22ms

oct. 02 14:35:32 Mini-labo2-firewall systemd[1]: Starting nftables.service - nftables...
oct. 02 14:35:32 Mini-labo2-firewall systemd[1]: Finished nftables.service - nftables.
```

Ensuite, nous créons un fichier de configuration : **touch /etc/nftables.conf**.

Ce fichier contient les règles du firewall.

Dans ce projet, nous décidons de :

- Autoriser le protocole **ICMP (ping)** pour tester la connectivité

```
flush ruleset

table inet filter {
    chain input {
        type filter hook input priority filter;

        #Autorisation loopback (localhost)
        iif "lo" accept

        #Autorisation des connexions établies
        ct state established,related accept

        #Autorisation du protocole ICMP (ping)
        ip protocol icmp accept
    }
}
```

- Autoriser **SSH** pour permettre l'accès distant

```
#Autorisation de SSH (port 22)
tcp dport 22 accept
```

- Bloquer le trafic **HTTP** afin de tester le filtrage

```
#Blocage de HTTP (port 80)
tcp dport 80 drop

#Blocage du reste
drop
}

chain forward {
    type filter hook forward priority filter;

    policy accept;
}

chain output {
    type filter hook output priority filter;

    policy accept;
}
}
```

Pour appliquer la configuration : `nft -f /etc/nftables.conf`.

Cette commande recharge les règles et permet d'assurer que les modifications sont prises en compte.

Enfin, nous redémarrons le service et vérifions que les règles sont bien actives.

```
user@Mini-labo2-firewall:~$ sudo systemctl restart nftables
[sudo] Mot de passe de user :
```

```
user@Mini-labo2-firewall:~$ sudo nft list ruleset
table inet filter {
    chain input {
        type filter hook input priority filter; policy accept;
        iif "lo" accept
        ct state established,related accept
        ip protocol icmp accept
        tcp dport 22 accept
        tcp dport 80 drop
        drop
    }

    chain forward {
        type filter hook forward priority filter; policy accept;
    }

    chain output {
        type filter hook output priority filter; policy accept;
    }
}
```

3- Configuration des adresses IP :

La machine routeur/firewall possède deux adresses IP :

- Une pour le réseau de la VM1
- Une pour le réseau de la VM2

```
user@Mini-labo2-firewall:~$ sudo ip addr add 192.168.20.1/24 dev ens36
user@Mini-labo2-firewall:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:f0:72:43 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.10.1/24 scope global ens33
        valid_lft forever preferred_lft forever
3: ens36: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:f0:72:4d brd ff:ff:ff:ff:ff:ff
    altname enp2s4
    inet 192.168.20.1/24 scope global ens36
        valid_lft forever preferred_lft forever
```

Chaque machine virtuelle doit être configurée avec :

- Une adresse IP appartenant à son sous-réseau

```
user@Mini-labo-VM1:~$ sudo ip addr add 192.168.10.10/24 dev ens33
[sudo] Mot de passe de user :
user@Mini-labo-VM1:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:42:7d:f1 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.10.10/24 scope global ens33
        valid_lft forever preferred_lft forever
```

```
user@Mini-labo-VM2:~$ sudo ip addr add 192.168.20.10/24 dev ens33
[sudo] Mot de passe de user :
user@Mini-labo-VM2:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:bb:40:18 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.20.10/24 scope global ens33
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:febb:4018/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

- Une passerelle correspondant à l'interface du routeur dans ce réseau

```
user@Mini-labo-VM1:~$ sudo ip route add default via 192.168.10.1
user@Mini-labo-VM1:~$ ip route
default via 192.168.10.1 dev ens33
192.168.10.0/24 dev ens33 proto kernel scope link src 192.168.10.10
```

```
user@Mini-labo2-firewall:~$ sudo ip addr add 192.168.20.1/24 dev ens36
user@Mini-labo2-firewall:~$ ip route
192.168.10.0/24 dev ens33 proto kernel scope link src 192.168.10.1
192.168.20.0/24 dev ens36 proto kernel scope link src 192.168.20.1
```

```
user@Mini-labo-VM2:~$ sudo ip route add default via 192.168.20.1
user@Mini-labo-VM2:~$ ip route
default via 192.168.20.1 dev ens33
192.168.20.0/24 dev ens33 proto kernel scope link src 192.168.20.10
```

Cela permet d'assurer la communication entre les deux sous-réseaux via le routeur.

Résultat :

1- Test avec ping :

Nous testons la connectivité entre les machines avec la commande **ping**.

Les résultats permettent de vérifier que la communication entre les deux sous-réseaux fonctionne correctement.

```
user@Mini-labo-VM1:~$ ping 192.168.10.1
PING 192.168.10.1 (192.168.10.1) 56(84) bytes of data.
64 bytes from 192.168.10.1: icmp_seq=1 ttl=64 time=0.715 ms
64 bytes from 192.168.10.1: icmp_seq=2 ttl=64 time=1.85 ms
64 bytes from 192.168.10.1: icmp_seq=3 ttl=64 time=2.13 ms
64 bytes from 192.168.10.1: icmp_seq=4 ttl=64 time=1.85 ms
^C
--- 192.168.10.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3039ms
rtt min/avg/max/mdev = 0.715/1.634/2.125/0.542 ms
user@Mini-labo-VM1:~$ ping 192.168.20.1
PING 192.168.20.1 (192.168.20.1) 56(84) bytes of data.
64 bytes from 192.168.20.1: icmp_seq=1 ttl=64 time=0.476 ms
64 bytes from 192.168.20.1: icmp_seq=2 ttl=64 time=1.57 ms
64 bytes from 192.168.20.1: icmp_seq=3 ttl=64 time=1.36 ms
64 bytes from 192.168.20.1: icmp_seq=4 ttl=64 time=1.12 ms
^C
--- 192.168.20.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3030ms
rtt min/avg/max/mdev = 0.476/1.129/1.565/0.408 ms
user@Mini-labo-VM1:~$ ping 192.168.20.10
PING 192.168.20.10 (192.168.20.10) 56(84) bytes of data.
64 bytes from 192.168.20.10: icmp_seq=1 ttl=63 time=1.19 ms
64 bytes from 192.168.20.10: icmp_seq=2 ttl=63 time=2.43 ms
64 bytes from 192.168.20.10: icmp_seq=3 ttl=63 time=1.92 ms
64 bytes from 192.168.20.10: icmp_seq=4 ttl=63 time=3.05 ms
^C
--- 192.168.20.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 1.191/2.147/3.046/0.680 ms
```

```
user@Mini-labo-VM2:~$ ping 192.168.20.1
PING 192.168.20.1 (192.168.20.1) 56(84) bytes of data.
64 bytes from 192.168.20.1: icmp_seq=1 ttl=64 time=0.646 ms
64 bytes from 192.168.20.1: icmp_seq=2 ttl=64 time=3.18 ms
64 bytes from 192.168.20.1: icmp_seq=3 ttl=64 time=2.72 ms
64 bytes from 192.168.20.1: icmp_seq=4 ttl=64 time=2.86 ms
^C
--- 192.168.20.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 0.646/2.350/3.176/0.997 ms
user@Mini-labo-VM2:~$ ping 192.168.10.1
PING 192.168.10.1 (192.168.10.1) 56(84) bytes of data.
64 bytes from 192.168.10.1: icmp_seq=1 ttl=64 time=0.719 ms
64 bytes from 192.168.10.1: icmp_seq=2 ttl=64 time=1.99 ms
64 bytes from 192.168.10.1: icmp_seq=3 ttl=64 time=1.88 ms
64 bytes from 192.168.10.1: icmp_seq=4 ttl=64 time=1.93 ms
^C
--- 192.168.10.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3019ms
rtt min/avg/max/mdev = 0.719/1.630/1.991/0.527 ms
user@Mini-labo-VM2:~$ ping 192.168.10.10
PING 192.168.10.10 (192.168.10.10) 56(84) bytes of data.
64 bytes from 192.168.10.10: icmp_seq=1 ttl=63 time=1.36 ms
64 bytes from 192.168.10.10: icmp_seq=2 ttl=63 time=3.34 ms
64 bytes from 192.168.10.10: icmp_seq=3 ttl=63 time=3.89 ms
64 bytes from 192.168.10.10: icmp_seq=4 ttl=63 time=5.01 ms
^C
--- 192.168.10.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 1.362/3.398/5.010/1.321 ms
```

2- Test SSH :

A- Installation de SSH :

Pour installer SSH, les machines sont temporairement configurées en mode NAT afin d'accéder à Internet.

```
user@Mini-labo-VM1:~$ sudo apt update
[sudo] Mot de passe de user :
Atteint :1 http://security.debian.org/debian-security bookworm-security InRelease
Atteint :2 http://deb.debian.org/debian bookworm InRelease
Atteint :3 http://deb.debian.org/debian bookworm-updates InRelease
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
6 paquets peuvent être mis à jour. Exécutez « apt list --upgradable » pour les voir.
user@Mini-labo-VM1:~$ sudo apt install openssh-server -y
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Les paquets supplémentaires suivants seront installés :
  openssh-sftp-server runit-helper
```

```
user@Mini-labo-VM2:~$ sudo apt update
[sudo] Mot de passe de user :
Atteint :1 http://security.debian.org/debian-security bookworm-security InRelease
Atteint :2 http://deb.debian.org/debian bookworm InRelease
Atteint :3 http://deb.debian.org/debian bookworm-updates InRelease
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
6 paquets peuvent être mis à jour. Exécutez « apt list --upgradable » pour les voir.
user@Mini-labo-VM2:~$ sudo apt install openssh-server -y
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Les paquets supplémentaires suivants seront installés :
  openssh-sftp-server runit-helper
```

Une fois l'installation terminée, elles sont remises dans leurs LAN respectifs.

B- Test :

Nous testons ensuite la connexion SSH entre les machines afin de vérifier que ce protocole est bien autorisé par le firewall.

```
user@Mini-labo-VM1:~$ ssh user@192.168.20.10
The authenticity of host '192.168.20.10 (192.168.20.10)' can't be established.
ED25519 key fingerprint is SHA256:e+UY9Gt6ZxNec5WxcE4winwJnrt+ouN36jZwskLEeiQ.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.20.10' (ED25519) to the list of known hosts.
user@192.168.20.10's password:
Linux Mini-labo-VM2 6.1.0-39-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.148-1 (2025-08-26) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
user@Mini-labo-VM2:~$ ping 192.168.10.10
PING 192.168.10.10 (192.168.10.10) 56(84) bytes of data.
64 bytes from 192.168.10.10: icmp_seq=1 ttl=63 time=0.956 ms
64 bytes from 192.168.10.10: icmp_seq=2 ttl=63 time=1.68 ms
64 bytes from 192.168.10.10: icmp_seq=3 ttl=63 time=1.87 ms
^C
--- 192.168.10.10 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 0.956/1.501/1.872/0.393 ms
```

```
user@Mini-labo-VM2:~$ ssh user@192.168.10.10
The authenticity of host '192.168.10.10 (192.168.10.10)' can't be established.
ED25519 key fingerprint is SHA256:z0jqY/rac/L3fNsp9B3mpzs16JfIFLUA+G+Ao0o8Lktg.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.10.10' (ED25519) to the list of known hosts.
user@192.168.10.10's password:
Linux Mini-labo-VM1 6.1.0-40-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.153-1 (2025-09-20) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
```

3- Test HTTP :

A- Installation de curl :

Comme précédemment, les machines passent temporairement en mode NAT pour installer l'outil curl, puis reviennent dans leurs LAN respectifs.

```
user@Mini-labo-VM1:~$ sudo apt install curl
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
```

```
user@Mini-labo-VM2:~$ sudo apt install curl
[sudo] Mot de passe de user :
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
```

B- Test :

Le test HTTP permet de vérifier que ce protocole est bien bloqué par le firewall.

```
user@Mini-labo-VM1:~$ curl http://192.168.20.10
curl: (7) Failed to connect to 192.168.20.10 port 80 after 1 ms: Couldn't connect to server
user@Mini-labo-VM1:~$

user@Mini-labo-VM2:~$ curl http://192.168.10.10
curl: (7) Failed to connect to 192.168.10.10 port 80 after 2 ms: Couldn't connect to server
user@Mini-labo-VM2:~$
```

Conclusion :

Ce projet a permis de mettre en place deux sous-réseaux distincts reliés par une machine virtuelle jouant le rôle de routeur/firewall.

L'utilisation de nftables a permis de configurer des règles de filtrage afin de contrôler les flux réseau entre les deux LAN.

Les tests réalisés ont permis de vérifier l'impact des règles mise en place sur la communication entre les machines.

Ce projet met en évidence l'importance d'un firewall dans la sécurisation et la gestion du trafic réseau.

Il reproduit également une architecture simplifiée de réseau d'entreprise.